

Reviewer Pack — Fekete Capacity of Laplacian Spectrum Upper-Bounds Max-Cut S...

Entry 999ba4b45fab · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-17 20:47:59 UTC

Fekete Capacity of Laplacian Spectrum Upper-Bounds Max-Cut SoS-2 Gap

Entry ID: 999ba4b45fab **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-17 20:47:59 UTC

1. Conjecture

Field A (mathematical branch): Logarithmic potential theory — Fekete transfinite diameter / Van-dermonde capacity of finite point sets on $\mathbb{R}$, in the Saff-Totik / Ransford / Hardin-Saff tradition (equilibrium measures, Robin constants, weighted log-energy). This corner of classical analysis has essentially no presence in SoS / CSP integrality-gap literature (arXiv ‘transfinite diameter’ AND ‘sum of squares’ OR ‘Lasserre’ OR ‘Max-Cut’) returns 0 direct hits, with <5 adjacent random-matrix papers on Wigner-type spectral statistics). **Field B** (complexity object): Degree-2 Sum-of-Squares / Lasserre integrality gap for Max-Cut on small graphs, accessed through the Delorme-Poljak eigenvalue upper bound $\text{SDP}_2(G) \leq (n/4) \cdot \lambda_n(L_G)$ (which coincides with degree-2 SoS for vertex-transitive graphs and is a valid SoS-2 upper bound in general). This is the Goemans-Williamson / Schoenebeck-Khot-Vishnoi / Barak-Steurer regime targeted by SOS_HIERARCHY.

Statement:

For a connected graph G on n vertices with Laplacian L_G having nontrivial eigenvalues $0 < \lambda_2 \leq \dots \leq \lambda_n$, let $\tilde{\sigma}(G) = \{\lambda_2/\lambda_n, \dots, \lambda_{n-1}/\lambda_n, 1\}$ be the spectrum rescaled into $(0,1]$, and define the normalized Fekete capacity $\text{Cap}(G) := (\prod_{2 \leq i < j \leq n} |\tilde{\lambda}_i - \tilde{\lambda}_j|)^{2/((n-1)(n-2))} \in [0,1]$ (the transfinite diameter of the rescaled spectrum, taken as 0 when any pair coincides exactly). Let $\text{gap}(G) := n \cdot \lambda_n / (4 \cdot \text{MaxCut}(G)) - 1 \geq 0$ be the Delorme-Poljak / SoS-2 multiplicative integrality gap. Conjecture: there is an absolute constant $C \leq 1$ such that for every connected graph G on $6 \leq n \leq 16$ vertices, $\text{gap}(G) \leq C \cdot \text{Cap}(G)$; equivalently, any nonzero SoS-2 integrality gap forces the rescaled Laplacian spectrum to have positive transfinite diameter, with the gap controlled linearly by it. A single graph G with $\text{gap}(G) > C \cdot \text{Cap}(G)$ by additive margin > 0.01 refutes the conjecture.

Rationale (proposer’s reasoning):

Bipartite-like graphs ($K_{n/2, n/2}$, balanced complete multipartite) saturate the Delorme-Poljak bound ($\text{gap} = 0$) AND have multiply-repeated Laplacian eigenvalues ($\text{Cap} = 0$), while expander 3-regular graphs have a well-spread spectrum ($\text{Cap} > 0$) AND a positive gap (≈ 0.087); the conjecture quantifies this co-occurrence by bounding the SoS-2 gap above by the geometric-mean spectral-gap product, a classical potential-theoretic invariant from Saff-Totik that has never been used as a hardness functional. The invariant is computed on the *graph* (a structural object) — not on a truth table — so it is shielded from NATURAL_PROOFS; it uses logarithms (non-ring), so it is shielded from ALGEBRIZATION; and it concerns concrete graphs, not oracles, so it is shielded from RELATIVIZATION. The

bridge sharpens existing Delorme–Poljak / Goemans–Williamson analysis by replacing the single λ_n with the full Vandermonde-style multiplicative structure of $\sigma(L_G)$.

Taxonomy category: SOS_HIERARCHY (status at proposal time: alive)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): aaebe95b325f536e

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

Across 30 seeds $\times n \in \{8, 12, 16\} \times 5$ ensembles (=450 graphs), for each connected G test $\text{gap}(G) \leq 1 \cdot \text{Cap}(G) + 0.01$. SUPPORTED iff $\geq 95\%$ of instances satisfy this AND every ensemble contributes ≥ 80 tested graphs. FALSIFIED iff any single instance has $\text{gap}(G) - \text{Cap}(G) > 0.01$.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	0.97	SAFE	SAFE
NATURAL PROOFS	SAFE	0.95	SAFE	SAFE
ALGEBRIZATION	SAFE	0.95	SAFE	SAFE
KARP_LIPTON	SAFE	0.96	SAFE	SAFE

4. Novelty audit

Verdict: NOVEL against 9 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - Delorme Poljak eigenvalue bound Max-Cut Laplacian spectrum integrality gap - transfinite diameter Fekete points graph Laplacian spectrum eigenvalues - Lasserre sum of squares Max-Cut Laplacian eigenvalue gap vertex-transitive

Top relevant hits considered: - [<http://arxiv.org/abs/1107.1630v3>] On the Integrality Gap of the Subtour LP for the 1,2-TSP - [<http://arxiv.org/abs/1207.5696v1>] Beyond Max-Cut: λ -Extendible Properties Parameterized Above the Poljak-Turzík Bound - [<http://arxiv.org/abs/2205.10968v4>] Eigenvalue bounds of the Kirchhoff Laplacian - [<http://arxiv.org/abs/1210.8059v3>] Equidistribution estimates for Fekete points on complex manifolds - [<http://arxiv.org/abs/1301.6896v1>] Laplacians on periodic discrete graphs - [<http://arxiv.org/abs/1806.00072v2>] On graph Laplacian eigenvectors with components in $\{-1, 0, 1\}$ - [<http://arxiv.org/abs/1505.06840v3>] A max-cut formulation of 0/1 programs - [<http://arxiv.org/abs/2306.05306v3>] Vertex isoperimetry on signed graphs and spectra of non-bipartite Cayley and Cayley sum graphs

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, $\leq 90s$ wall time per cycle **Execution:** rc=0, elapsed=0.1s

5.1 Generated Python source

```
import sys
import random
import math
```

```

import itertools
import json

def matrix_multiply(A, B):
    n = len(A)
    result = [[0.0 for _ in range(n)] for _ in range(n)]
    for i in range(n):
        for j in range(n):
            for k in range(n):
                result[i][j] += A[i][k] * B[k][j]
    return result

def matrix_subtract(A, B):
    n = len(A)
    return [[A[i][j] - B[i][j] for j in range(n)] for i in range(n)]

def jacobi_rotation(A, max_iterations=30):
    n = len(A)
    V = [[0.0 if i != j else 1.0 for j in range(n)] for i in range(n)]
    for _ in range(max_iterations):
        max_off_diag = 0.0
        p, q = 0, 1
        for i in range(n):
            for j in range(i + 1, n):
                if abs(A[i][j]) > max_off_diag:
                    max_off_diag = abs(A[i][j])
                    p, q = i, j
        if max_off_diag < 1e-10:
            break
        if A[p][p] == A[q][q]:
            theta = math.pi / 4
        else:
            theta = 0.5 * math.atan(2 * A[p][q] / (A[p][p] - A[q][q]))
        c = math.cos(theta)
        s = math.sin(theta)
        for i in range(n):
            new_p = c * A[p][i] - s * A[q][i]
            new_q = s * A[p][i] + c * A[q][i]
            A[p][i] = new_p
            A[q][i] = new_q
        for i in range(n):
            new_p = c * A[i][p] - s * A[i][q]
            new_q = s * A[i][p] + c * A[i][q]
            A[i][p] = new_p
            A[i][q] = new_q
        for i in range(n):
            new_p = c * V[i][p] - s * V[i][q]
            new_q = s * V[i][p] + c * V[i][q]
            V[i][p] = new_p
            V[i][q] = new_q
    eigenvalues = [A[i][i] for i in range(n)]
    return eigenvalues, V

def compute_laplacian_eigenvalues(adj_matrix):
    n = len(adj_matrix)
    degree_matrix = [[0.0 for _ in range(n)] for _ in range(n)]

```

```

    for i in range(n):
        degree_matrix[i][i] = sum(adj_matrix[i])
    laplacian = matrix_subtract(degree_matrix, adj_matrix)
    eigenvalues, _ = jacobi_rotation(laplacian)
    eigenvalues.sort()
    return [e for e in eigenvalues if abs(e) > 1e-10]

def compute_capacity(eigenvalues):
    if len(eigenvalues) < 2:
        return 0.0
    max_eig = max(eigenvalues)
    if max_eig == 0:
        return 0.0
    rescaled = [e / max_eig for e in eigenvalues if e > 0]
    if len(rescaled) < 2:
        return 0.0
    product = 1.0
    for i in range(len(rescaled)):
        for j in range(i + 1, len(rescaled)):
            product *= abs(rescaled[i] - rescaled[j])
    denominator = (len(rescaled) - 1) * (len(rescaled) - 2)
    if denominator == 0:
        return 0.0
    return product ** (2.0 / denominator)

def generate_random_3_regular(n):
    if n % 2 != 0:
        return None
    edges = []
    stubs = list(range(n)) * 3
    random.shuffle(stubs)
    while stubs:
        u = stubs.pop()
        v = stubs.pop()
        if u != v and (u, v) not in edges and (v, u) not in edges:
            edges.append((u, v))
    adj_matrix = [[0 for _ in range(n)] for _ in range(n)]
    for u, v in edges:
        adj_matrix[u][v] = 1
        adj_matrix[v][u] = 1
    return adj_matrix

def generate_erdos_renyi(n, p):
    adj_matrix = [[0 for _ in range(n)] for _ in range(n)]
    for i in range(n):
        for j in range(i + 1, n):
            if random.random() < p:
                adj_matrix[i][j] = 1
                adj_matrix[j][i] = 1
    return adj_matrix

def generate_complete_bipartite(n):
    k = n // 2
    adj_matrix = [[0 for _ in range(n)] for _ in range(n)]
    for i in range(k):
        for j in range(k, n):

```

```

        adj_matrix[i][j] = 1
        adj_matrix[j][i] = 1
    return adj_matrix

def generate_random_union_of_cliques(n):
    a = random.randint(2, n - 2)
    adj_matrix = [[0 for _ in range(n)] for _ in range(n)]
    for i in range(a):
        for j in range(i + 1, a):
            adj_matrix[i][j] = 1
            adj_matrix[j][i] = 1
    for i in range(a, n):
        for j in range(i + 1, n):
            adj_matrix[i][j] = 1
            adj_matrix[j][i] = 1
    u = random.randint(0, a - 1)
    v = random.randint(a, n - 1)
    adj_matrix[u][v] = 1
    adj_matrix[v][u] = 1
    return adj_matrix

def generate_cycle_with_chord(n):
    adj_matrix = [[0 for _ in range(n)] for _ in range(n)]
    for i in range(n):
        adj_matrix[i][(i + 1) % n] = 1
        adj_matrix[(i + 1) % n][i] = 1
    u = random.randint(0, n - 1)
    v = random.randint(0, n - 1)
    if u != v and abs(u - v) > 1 and abs(u - v) < n - 1:
        adj_matrix[u][v] = 1
        adj_matrix[v][u] = 1
    return adj_matrix

def is_connected(adj_matrix):
    n = len(adj_matrix)
    visited = [False] * n
    stack = [0]
    visited[0] = True
    while stack:
        u = stack.pop()
        for v in range(n):
            if adj_matrix[u][v] and not visited[v]:
                visited[v] = True
                stack.append(v)
    return all(visited)

def compute_max_cut(adj_matrix):
    n = len(adj_matrix)
    max_cut_size = 0
    for mask in range(1, 1 << n):
        cut_size = 0
        for u in range(n):
            for v in range(u + 1, n):
                if ((mask >> u) & 1) != ((mask >> v) & 1) and adj_matrix[u][v]:
                    cut_size += 1
        if cut_size > max_cut_size:

```

```

        max_cut_size = cut_size
    return max_cut_size

def run_trial(seed):
    random.seed(seed)
    n_values = [8, 12, 16]
    ensemble_generators = [
        generate_random_3_regular,
        lambda n: generate_erdos_renyi(n, 0.3),
        generate_complete_bipartite,
        generate_random_union_of_cliques,
        generate_cycle_with_chord
    ]
    metric_values = []
    instances_tested = 0
    conjecture_holds = True
    counterexample = ""
    for n in n_values:
        for generator in ensemble_generators:
            for _ in range(5):
                adj_matrix = generator(n)
                if adj_matrix is None or not is_connected(adj_matrix):
                    continue
                eigenvalues = compute_laplacian_eigenvalues(adj_matrix)
                if len(eigenvalues) < 2:
                    continue
                cap = compute_capacity(eigenvalues)
                max_cut = compute_max_cut(adj_matrix)
                if max_cut == 0:
                    continue
                gap = n * eigenvalues[-1] / (4 * max_cut) - 1
                if gap > 1 * cap + 0.01:
                    conjecture_holds = False
                    counterexample = f"n={n}, gap={gap}, cap={cap}, eigenvalues={eigenvalues}, max_cut={max_cut}"
                    break
                metric_values.append(gap - cap)
                instances_tested += 1
            if not conjecture_holds:
                break
        if not conjecture_holds:
            break
    if conjecture_holds and len(metric_values) > 0:
        mean_metric = sum(metric_values) / len(metric_values)
        std_metric = math.sqrt(sum((x - mean_metric) ** 2 for x in metric_values) / len(metric_values))
    else:
        mean_metric = 0.0
        std_metric = 0.0
    return {
        "metric_name": "gap_minus_cap",
        "metric_value": mean_metric,
        "metric_std": std_metric,
        "instances_tested": instances_tested,
        "conjecture_holds": conjecture_holds,
        "counterexample": counterexample
    }
}

```

```

if __name__ == "__main__":
    seeds = [int(arg) for arg in sys.argv[1:]] if len(sys.argv) > 1 else [2, 3, 5, 7, 11, 13, 17,
    results = []
    for seed in seeds:
        result = run_trial(seed)
        result["seed"] = seed
        print(f"TRIAL: {json.dumps(result)}")
        results.append(result)
    total_instances = sum(r["instances_tested"] for r in results)
    if total_instances == 0:
        print("RESULT: INCONCLUSIVE reason=no_instances_tested")
        sys.exit(0)
    mean_metric = sum(r["metric_value"] for r in results) / len(results)
    std_metric = math.sqrt(sum((r["metric_value"] - mean_metric) ** 2 for r in results) / len(results))
    support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)
    if support_fraction >= 0.95:
        print(f"RESULT: SUPPORTED mean={mean_metric:.4f} std={std_metric:.4f} support_fraction={support_fraction:.4f}")
    else:
        falsified = any(r["counterexample"] for r in results)
        if falsified:
            first_failing_seed = next(r["seed"] for r in results if r["counterexample"])
            first_counterexample = next(r["counterexample"] for r in results if r["counterexample"] == first_failing_seed)
            print(f"RESULT: FALSIFIED counterexample=\"{first_counterexample}\" first_failing_seed={first_failing_seed}")
        else:
            print("RESULT: INCONCLUSIVE reason=insufficient_support")

```

6. Per-seed results

Seed	Metric value	Holds?	Counterexample
11	0.0	□	n=8, gap=0.29829009509915916, cap=0.18968001834167775, eigenvalues=[0.3860139185
23	0.0	□	n=8, gap=0.08637778554549169, cap=0.0, eigenvalues=[1.2105182466244415, 1.384438
37	0.0	□	n=8, gap=0.40751737339938776, cap=0.08250714676059329, eigenvalues=[0.5857864376
53	0.0	□	n=8, gap=0.146608756192514, cap=0.0, eigenvalues=[0.41356497522994506, 1.0221610
71	0.0	□	n=8, gap=0.3545403757369132, cap=0.1770536482392048, eigenvalues=[1.076738478328

Seed	Metric value	Holds?	Counterexample
89	0.0	□	n=8, gap=0.24858932172373738, cap=0.17795982459697166, eigenval- ues=[0.5007238480
103	0.0	□	n=8, gap=0.22353198571165, cap=0.17944518268547696, eigenval- ues=[0.2805768657845
127	0.0	□	n=8, gap=0.2893190272918611, cap=0.1680349499993342, eigenval- ues=[1.183209279467
149	0.0	□	n=8, gap=0.2184466913069627, cap=0.11501094742139917, eigenval- ues=[0.58578643762
167	0.0	□	n=8, gap=0.08513615511612893, cap=0.03676067202365689, eigenval- ues=[0.5857864376
191	0.0	□	n=8, gap=0.228280467467467, cap=0.1775420094450226, eigenval- ues=[0.5572737406860
211	0.0	□	n=8, gap=0.058686417410234615, cap=0.0, eigenval- ues=[0.5857864376269052, 0.58578
233	0.0	□	n=8, gap=0.23648197904262958, cap=0.20385595440143575, eigenval- ues=[0.2026980972
257	0.0	□	n=8, gap=0.016464333447161472, cap=0.0, eigenval- ues=[0.9829952717948447, 1.0, 1.
277	0.0	□	n=8, gap=0.18501877317122917, cap=0.0, eigenval- ues=[0.5857864376269052, 0.585786
311	0.0	□	n=8, gap=0.24761759995471944, cap=0.0, eigenval- ues=[0.5857864376269052, 0.585786

Seed	Metric value	Holds?	Counterexample
347	0.0	□	n=8, gap=0.21684366770442143, cap=0.16304596512606961, eigenval- ues=[0.4539061334
389	0.0	□	n=8, gap=0.18277321598647323, cap=0.12684841709181288, eigenval- ues=[0.3346611905
421	0.0	□	n=8, gap=0.2193619865618197, cap=0.0, eigenval- ues=[0.33327201670721396, 0.585786
463	0.0	□	n=8, gap=0.2193619865618197, cap=0.0, eigenval- ues=[0.3332720167072124, 0.5857864
503	0.0	□	n=8, gap=0.23107006719019907, cap=0.15033210827607918, eigenval- ues=[0.9565520008
547	0.0	□	n=8, gap=0.25873239100756606, cap=0.16884515430408875, eigenval- ues=[0.6974768385
593	0.0	□	n=8, gap=0.21684366770442143, cap=0.1630459651260696, eigenval- ues=[0.45390613346
631	0.0	□	n=8, gap=0.2265112576258903, cap=0.18985438241853905, eigenval- ues=[0.21001488944
677	0.0	□	n=8, gap=0.2193619865618197, cap=0.03796655471110021, eigenval- ues=[0.33327201670

Aggregate statistics:

Statistic	Value
n_seeds	25
metric_mean	0.0
metric_std	0.0
metric_ci95_half	0.0

Statistic	Value
metric_min	0.0
metric_max	0.0
support_fraction	0.0

7. Test stdout (last 2KB)

```
6, "conjecture_holds": false, "counterexample": "n=8, gap=0.228280467467467, cap=0.1775420094450226,
TRIAL: {"metric_name": "gap_minus_cap", "metric_value": 0.0, "metric_std": 0.0, "instances_tested": 1}
TRIAL: {"metric_name": "gap_minus_cap", "metric_value": 0.0, "metric_std": 0.0, "instances_tested": 1}
TRIAL: {"metric_name": "gap_minus_cap", "metric_value": 0.0, "metric_std": 0.0, "instances_tested": 1}
TRIAL: {"metric_name": "gap_minus_cap", "metric_value": 0.0, "metric_std": 0.0, "instances_tested": 1}
TRIAL: {"metric_name": "gap_minus_cap", "metric_value": 0.0, "metric_std": 0.0, "instances_tested": 1}
```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

The data does NOT support the conjecture — every per-seed entry shows holds=false with support_fraction=0.0. Seed 23 alone is a decisive refutation: gap=0.0864 with Cap=0.0 (eigenvalue coincidence in the rescaled spectrum), so $C \cdot \text{Cap} = 0 < \text{gap}$ by margin > 0.08 , which exceeds the 0.01 additive threshold the statement specifies. Seed 53 is identical (gap=0.147, Cap=0). The conjecture fails specifically at the construction’s degenerate boundary: graphs with repeated Laplacian eigenvalues (common for ve

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

Safety rail: critic_challenge_falsified | original: Multiple instances violate $\text{gap}(G) \leq \text{Cap}(G) + 0.01$ by large margins; e.g., $n=8$ with $\text{gap} \approx 0.2587$, $\text{Cap} \approx 0.1688$ (margin ≈ 0.09), and degenerate cases like se | next: Replace Fekete capacity with a degeneracy-robust separation surrogate (e.g., $\min$ pairwise gap regularized as $\log(1/(\epsilon + \min_i |\tilde{\lambda}_i - \tilde{\lambda}_j|))$) or restrict to graphs with simple Laplacian spectra before re-testing the gap bound.

11. Audit log (LLM calls)

Total LLM calls: 8

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	claude_max	opus	0	0	232324
2	preregistration	claude_max	opus	0	0	6137
3	novelty	claude_max	opus	0	0	3874
4	novelty	claude_max	opus	0	0	9475
5	test_gen	mistral	codestral-latest	0	0	319943
6	test_gen	mistral	codestral-latest	0	0	322961
7	critic	claude_max	opus	0	0	14485
8	judge	claude_max	opus	0	0	6902

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 916100 ms total latency. Provider mix: {'claude_max': 6, 'mistral': 2}

(full prompt+response transcripts available in research/audit/999ba4b45fab.jsonl)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in research/audit/999ba4b45fab.jsonl for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: research/pvsnp_sandbox/test_*.py (per-cycle)

Replay tarball: research/replay/999ba4b45fab.tar.gz (if generated)